

REST API Naming Conventions

A straightforward, standard reference guide for designing clean, predictable, and resource-oriented RESTful APIs.

1. Core URI Naming Rules

- **Use nouns, never verbs:** URIs identify resources, not actions. HTTP methods (`GET`, `POST`, etc.) express the operation.
 - Good: `GET /users`, `POST /orders`
 - Bad: `/getUsers`, `/createOrder`, `/deleteCustomer`
- **Use plural nouns for collections:** Plural endpoints keep conventions consistent across singular and collection queries.
 - Good: `/products` (collection), `/products/42` (single item)
 - Bad: `/product`, `/product/42`
- **Use lowercase and kebab-case:** URLs are case-sensitive on many servers. Avoid camelCase or snake_case in path segments.
 - Good: `/user-profiles`, `/order-items`
 - Bad: `/userProfiles`, `/order_items`
- **Omit file extensions:** Use standard HTTP `Accept` and `Content-Type` headers for content negotiation instead of suffixes.
 - Good: `GET /invoices/101` (Header: `Accept: application/pdf`)
 - Bad: `GET /invoices/101.pdf`
- **Avoid trailing slashes:** Trailing slashes introduce routing ambiguities and unnecessary redirects.
 - Good: `/customers`
 - Bad: `/customers/`

2. Standard HTTP Verbs & CRUD Mapping

Map resource operations strictly to standard HTTP methods rather than inventing custom action paths.

Method	URI Example	Action Description	Status Code
<code>GET</code>	<code>/orders</code>	Retrieve a list of orders	<code>200 OK</code>
<code>GET</code>	<code>/orders/101</code>	Retrieve details of order 101	<code>200 OK</code>
<code>POST</code>	<code>/orders</code>	Create a new order entity	<code>201 Created</code>
<code>PUT</code>	<code>/orders/101</code>	Completely replace order 101	<code>200 / 204</code>
<code>PATCH</code>	<code>/orders/101</code>	Partially update fields on order 101	<code>200 OK</code>
<code>DELETE</code>	<code>/orders/101</code>	Permanently delete order 101	<code>204 No Content</code>

3. Resource Relationships & Nesting

Nest sub-resources only when a child resource strictly exists within the context of a parent. Limit hierarchy to a maximum of two levels.

Pattern	Example	Usage
<code>/ {parents} / {id} / {children}</code>	<code>GET /authors/12/books</code>	Sub-collection belonging to parent
<code>/ {parents} / {id} / {children} / {id}</code>	<code>GET /authors/12/books/99</code>	Specific item in nested relationship
<code>/ {children} / {id}</code> (Flattened)	<code>GET /books/99</code>	Direct lookup when child ID is unique

Best Practice Tip: If a nested resource can be uniquely addressed by its own primary identifier, prefer top-level flattened routes (e.g. `/comments/89`) over deep nesting (e.g. `/users/3/posts/12/comments/89`).

4. Filtering, Sorting, and Pagination

Do not create dedicated endpoints to change representation or slice data. Use query parameters instead.

- **Filtering:** `GET /products?category=hardware&status=in-stock`
- **Sorting:** `GET /users?sort=created_at:desc` or `GET /users?sort=-created_at`
- **Offset Pagination:** `GET /items?offset=20&limit=10`
- **Page-based Pagination:** `GET /items?page=3&per_page=25`
- **Field Selection (Sparse Fieldsets):** `GET /users?fields=id,name,email`

5. Non-CRUD & Controller Operations

When an action is not a simple entity manipulation (e.g., executing a business workflow or state transition):

- **Sub-resource noun approach (Preferred):** Model the action as a creation of a new sub-entity.
 - *Example:* `POST /accounts/42/resets` (instead of `/resetPassword`)
 - *Example:* `POST /orders/101/cancellations` (instead of `/cancelOrder`)
- **Verb / Controller suffix (Alternative):** If a noun model is overly unnatural, append a clear verb action to the specific resource.
 - *Example:* `POST /jobs/99/cancel`
 - *Example:* `POST /transactions/505/refund`

6. Versioning

Place the major version directly in the URI path or provide it in standard request headers:

- **URI Path (Standard):** `https://api.example.com/v1/users`
- **Header-based (Alternative):** `Accept: application/vnd.company.v1+json`